Fraud Detection using Machine Learning

---

1. Aim

To develop a machine learning model for detecting fraudulent credit card transactions using data preprocessing, class imbalance handling (SMOTE), and performance evaluation metrics.

---

2. Problem Statement

Credit card fraud detection is a challenging task due to the highly imbalanced nature of transaction data. The objective is to identify fraudulent transactions accurately while minimizing false positives.

---

3. Dataset Description

The dataset used is the Credit Card Fraud Detection dataset from Kaggle. (creditcard.csv)

Dataset details:

- Total transactions: **284,807**
- Fraudulent transactions: **492**
- Features: **30 (V1–V28, Time, Amount)**

Note: PCA Transformation

> The features V1 to V28 are transformed using Principal Component Analysis (PCA). This is done to protect sensitive customer information due to legal and privacy concerns. The original features are not available, and PCA ensures data anonymization while retaining important patterns required for fraud detection.

Dataset Preview : df.head()

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22	V23	V24	V25	V26	V27	V28	Amount	Class
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	-0.189115	0.133558	-0.021053	149.62	0
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	0.125895	-0.008983	0.014724	2.69	0
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	-0.139097	-0.055353	-0.059752	378.66	0
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	-0.221929	0.062723	0.061458	123.50	0
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.206010	0.502292	0.219422	0.215153	69.99	0

5 rows × 31 columns

Dataset Information : df.info() & df.isnull.sum()

```
Shape: (284807, 31)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   Time    284807 non-null  float64
 1   V1      284807 non-null  float64
 2   V2      284807 non-null  float64
 3   V3      284807 non-null  float64
 4   V4      284807 non-null  float64
 5   V5      284807 non-null  float64
 6   V6      284807 non-null  float64
 7   V7      284807 non-null  float64
 8   V8      284807 non-null  float64
 9   V9      284807 non-null  float64
 10  V10     284807 non-null  float64
 11  V11     284807 non-null  float64
 12  V12     284807 non-null  float64
 13  V13     284807 non-null  float64
 14  V14     284807 non-null  float64
 15  V15     284807 non-null  float64
 16  V16     284807 non-null  float64
 17  V17     284807 non-null  float64
 18  V18     284807 non-null  float64
 19  V19     284807 non-null  float64
 20  V20     284807 non-null  float64
 21  V21     284807 non-null  float64
 22  V22     284807 non-null  float64
 23  V23     284807 non-null  float64
 24  V24     284807 non-null  float64
 25  V25     284807 non-null  float64
 26  V26     284807 non-null  float64
 27  V27     284807 non-null  float64
 28  V28     284807 non-null  float64
 29  Amount  284807 non-null  float64
 30  Class   284807 non-null  int64
dtypes: float64(30), int64(1)
```

```
df.isnull().sum()
```

	0
Time	0
V1	0
V2	0
V3	0
V4	0
V5	0
V6	0
V7	0
V8	0
V9	0
V10	0
V11	0
V12	0
V13	0
V14	0
V15	0
V16	0
V17	0
V18	0
V19	0
V20	0
V21	0
V22	0
V23	0
V24	0
V25	0
V26	0
V27	0
V28	0
Amount	0
Class	0

---

4. Data Preprocessing

- Missing values were checked → none found
- Duplicate records (~1081) were identified
- Feature scaling applied to **Amount**
- Dataset split using stratified sampling

Duplicate Count

```
print("Duplicates:", df.duplicated().sum())
```

```
Duplicates: 1081
```

Feature scaling to Amount

```
[9]
✓ 0s
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

df['Amount'] = scaler.fit_transform(df['Amount'].values.reshape(-1,1))
```

Dataset Split

```
[11]
✓ 0s
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

---

5. Handling Class Imbalance

The dataset is highly imbalanced:

- Non-fraud ≈ 284,000
- Fraud ≈ 492

Technique used:

- **SMOTE (Synthetic Minority Oversampling Technique)**

Partial SMOTE (sampling ratio = 0.1) was applied to increase fraud samples while maintaining realistic data distribution.

```
import pandas as pd

print("Before SMOTE:\n", y_train.value_counts())
print("\nAfter SMOTE:\n", pd.Series(y_train_smote).value_counts())
```

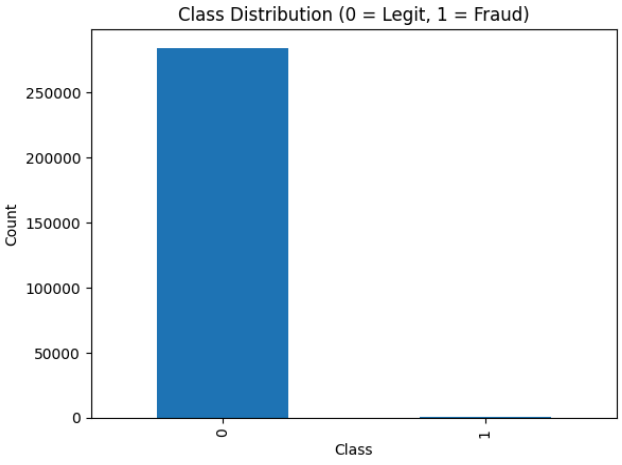
```
Before SMOTE:
 Class
0    227451
1       394
Name: count, dtype: int64
```

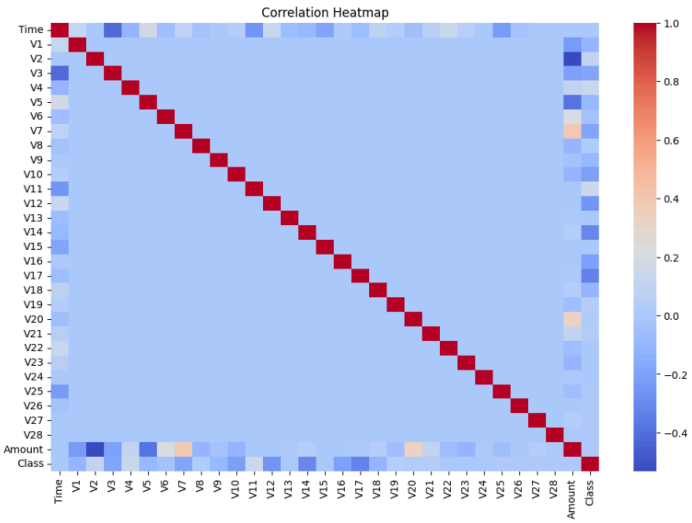
```
After SMOTE:
 Class
0    227451
1     22745
Name: count, dtype: int64
```

---

6. Exploratory Data Analysis (EDA)

- Severe class imbalance observed
- Correlation heatmap shows low correlation due to PCA features
- No strong multicollinearity





Most features (V1–V28) show very low correlation with each other as they are PCA-transformed variables. No strong multicollinearity is observed. The target variable (Class) shows weak correlation with individual features, indicating the need for machine learning models to capture complex patterns.

---

7. Model Implementation

The following models were implemented:

- Logistic Regression
- Decision Tree
- K-Nearest Neighbors
- Random Forest

[16]
✓ 0s

```
from sklearn.ensemble import RandomForestClassifier

models = {
    "Logistic Regression": LogisticRegression(max_iter=2000, random_state=42),
    "Decision Tree": DecisionTreeClassifier(max_depth=10, random_state=42),
    "K-Nearest Neighbors": KNeighborsClassifier(n_neighbors=7),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
}
```

---

8. Performance Evaluation Metrics

Models were evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC
- Confusion Matrix

Accuracy is not reliable due to class imbalance; recall is prioritized.

---

9. Results and Analysis

Classification Report

```
==========================================================
Training Logistic Regression
==========================================================

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     56864
           1       0.53      0.88      0.66        98

    accuracy                           1.00     56962
   macro avg       0.77      0.94      0.83     56962
weighted avg       1.00      1.00      1.00     56962
```

```
==========================================================
Training Decision Tree
==========================================================
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     56864
           1       0.45      0.87      0.59        98

    accuracy                           1.00     56962
   macro avg       0.72      0.93      0.80     56962
weighted avg       1.00      1.00      1.00     56962
```

```
==========================================================
Training K-Nearest Neighbors
==========================================================
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     56864
           1       0.44      0.52      0.48        98

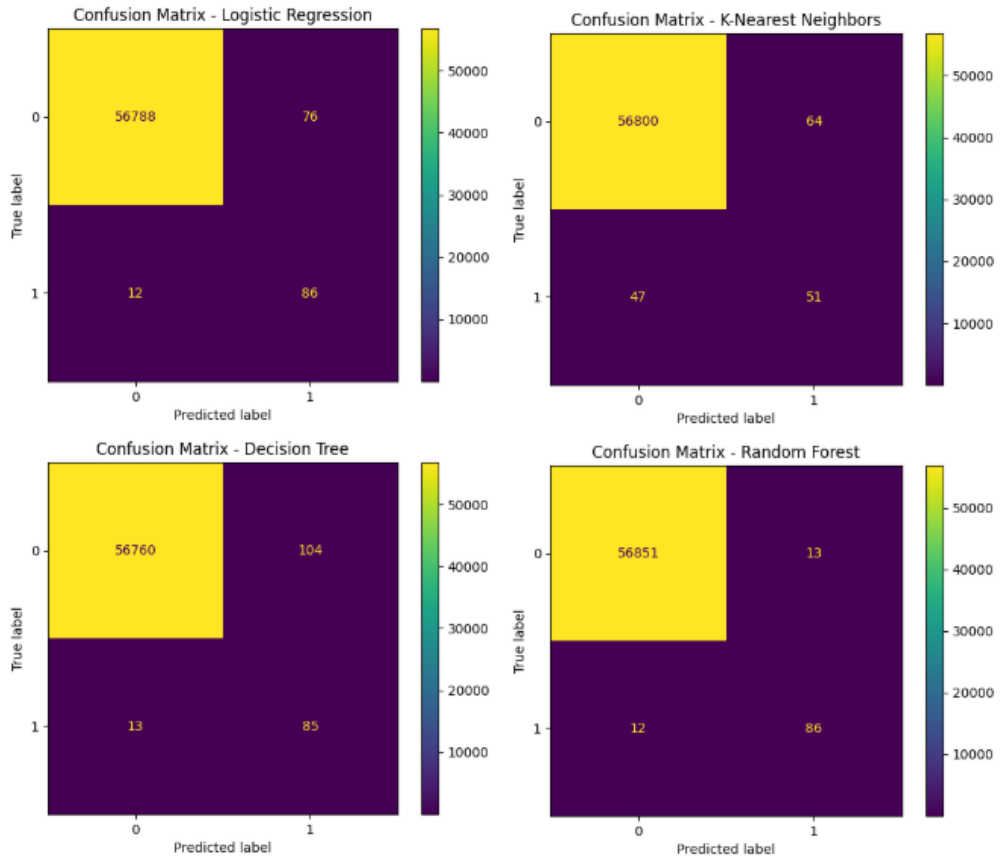
    accuracy                           1.00     56962
   macro avg       0.72      0.76      0.74     56962
weighted avg       1.00      1.00      1.00     56962
```

```
==========================================================
Training Random Forest
==========================================================
Classification Report:
              precision    recall  f1-score   support

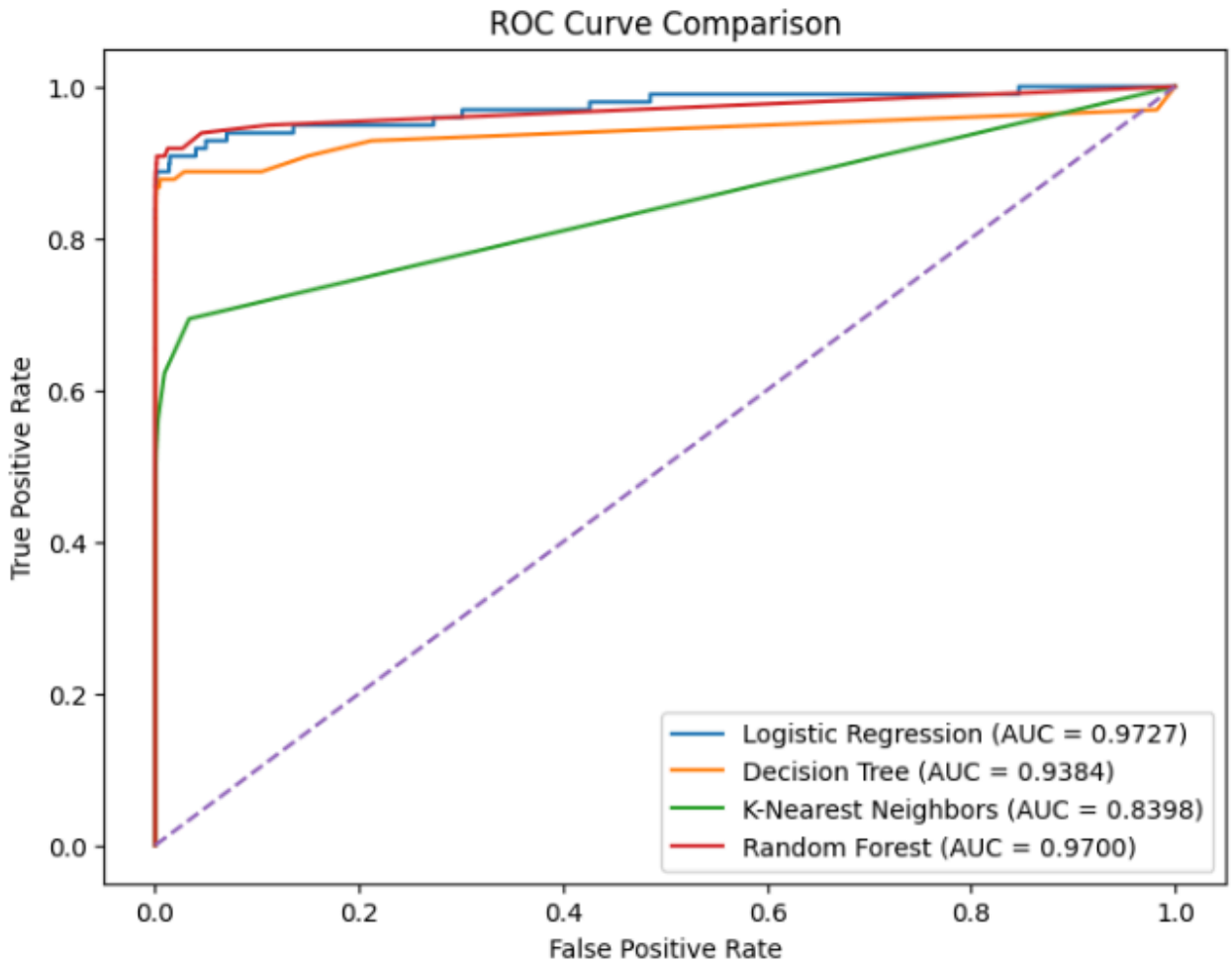
           0       1.00      1.00      1.00     56864
           1       0.87      0.88      0.87        98

    accuracy                           1.00     56962
   macro avg       0.93      0.94      0.94     56962
weighted avg       1.00      1.00      1.00     56962
```

Confusion Matrix



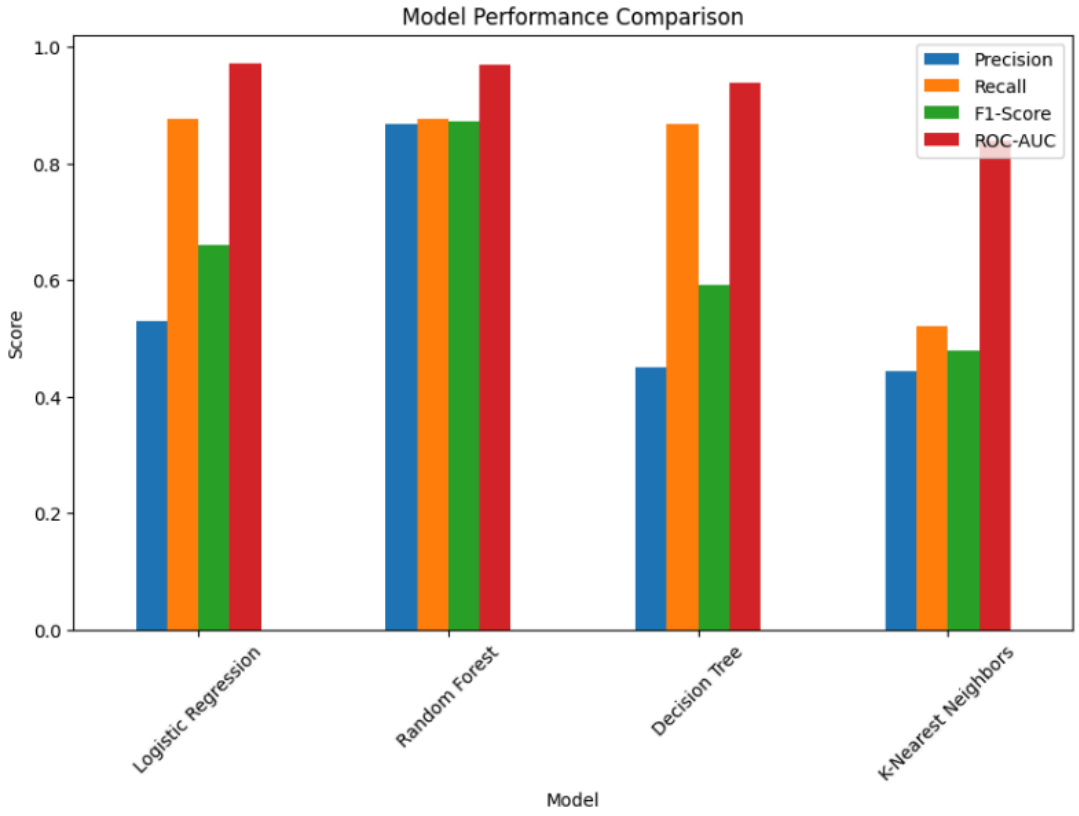
ROC Curve



Model Comparison Table

	Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
0	Logistic Regression	0.998455	0.530864	0.877551	0.661538	0.972722
3	Random Forest	0.999561	0.868687	0.877551	0.873096	0.969983
1	Decision Tree	0.997946	0.449735	0.867347	0.592334	0.938357
2	K-Nearest Neighbors	0.998051	0.443478	0.520408	0.478873	0.839776

Bar Chart Comparison



Analysis:

- Logistic Regression achieved high recall
- Decision Tree showed moderate performance
- KNN performed weaker due to high dimensionality
- Random Forest performed best overall

Random Forest provides the best balance between precision and recall.

---

10. Outlier Detection

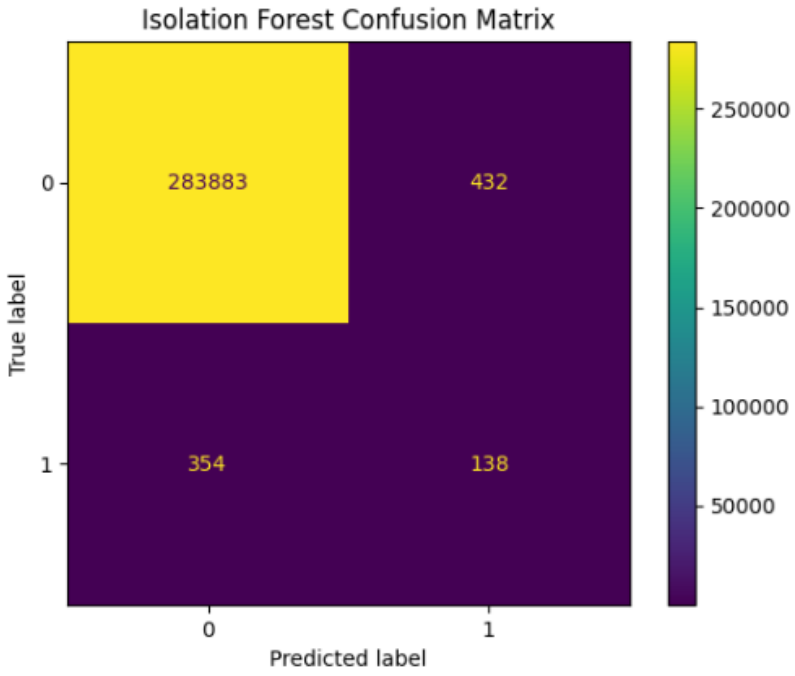
Isolation Forest was used as an unsupervised anomaly detection technique.

- Fraud cases treated as anomalies
- Lower performance compared to supervised models

Isolation Forest Classification Report

	precision	recall	f1-score	support
0	1.00	1.00	1.00	284315
1	0.24	0.28	0.26	492
accuracy			1.00	284807
macro avg	0.62	0.64	0.63	284807
weighted avg	1.00	1.00	1.00	284807

Isolation Forest Confusion Matrix



Analysis:

- Increasing contamination(from 0.001 to 0.002) improved recall
- Precision decreased → trade-off observed

Isolation Forest detects anomalies without labeled data but is less accurate than supervised models.

---

11. Conclusion

In this experiment, a comprehensive machine learning pipeline was developed to detect fraudulent credit card transactions. The dataset was first analyzed using descriptive statistics and visualization techniques, which revealed a highly imbalanced distribution between fraudulent and non-fraudulent transactions. Since the dataset features were transformed using PCA for privacy reasons, the relationships between variables were not directly interpretable, but sufficient patterns existed for model learning.

Data preprocessing included checking for missing values and duplicates, followed by feature scaling of the 'Amount' column. To address class imbalance, SMOTE (Synthetic Minority Oversampling Technique) was applied with a partial sampling strategy, which improved the representation of fraudulent transactions while maintaining a realistic data distribution.

Multiple machine learning models including Logistic Regression, Decision Tree, K-Nearest Neighbors, and Random Forest were implemented and evaluated. Due to the imbalanced nature of the dataset, accuracy was not considered a reliable metric; instead, precision, recall, F1-score, and ROC-AUC were used. Among all models, Random Forest achieved the best overall performance, providing a good balance between precision and recall. Logistic Regression also performed well in terms of recall, making it effective for identifying fraudulent transactions.

Additionally, an unsupervised anomaly detection approach using Isolation Forest was explored. While it demonstrated the ability to detect some fraudulent transactions without labeled data, its performance was lower compared to supervised models, highlighting the importance of labeled learning in such problems.

Overall, the experiment demonstrated that handling class imbalance and selecting appropriate evaluation metrics are crucial for fraud detection tasks. Recall was prioritized over precision, as missing fraudulent transactions can lead to significant financial losses. The study also showed that ensemble methods like Random Forest are highly effective for complex classification problems such as fraud detection.